Systèmes à évènements discrets

Cours

Compétences visées: A3-13, B1-03, B2-22, B2-23, B2-24, B2-25 v1.0

Lycée La Fayette – 21 Bd Robert Schuman – 63000 Clermont-Ferrand – Académie de Clermont-Ferrand



Compétences visées :

A3-13	Interpréter tout ou partie de l'évolution temporelle d'un système.
B1-03	Décrire l'évolution des grandeurs.
B2-22	Coder une information.
B2-23	Exprimer un fonctionnement par des équations logiques.
B2-24	Représenter tout ou partie de l'évolution temporelle.
B2-25	Décrire et compléter un algorithme représenté sous forme graphique.

Table des matières

1 Préambule **4**

2 Architecture des systèmes de commande **4**

2.1 Rôle d'un système de commande 4

2.2 Architecture 5

2.2.1 Structure d'un FPGA 6

2.2.2 Structure d'un micro-contrôleur 6

2.2.3 Les processeurs de signaux digitaux (DSP) 7

2.2.4 Vers les ordinateurs embarqués et l'architecture distribuée 7

2.3 Nature des informations 8

3 Traitement des informations binaires **8**

3.1 Système de numération 9

3.1.1 Bases 9

3.1.2 Changement de bases : transcodage 9

3.2 Codage de l'information 10

3.2.1 Code binaire naturel 11

3.2.2 Code décimal codé binaire (Code DCB) 11

3.2.3 Code binaire réfléchi ou code Gray 12

3.2.4 Code p parmi n 13

4 Logique combinatoire **13**

4.1 Définitions 13

4.1.1 Variables binaires 13

4.1.2 Système binaire 14

4.1.3 Systèmes combinatoire et séquentiel 14

4.2 Algèbre de Boole 15

4.3 Représentation des fonctions logiques 16

4.3.1 Table de vérité 17

4.3.2 Équations logiques 17

4.3.3 Logigrammes 18

4.3.4 Chronogramme 19

5 Logique séquentielle **20**

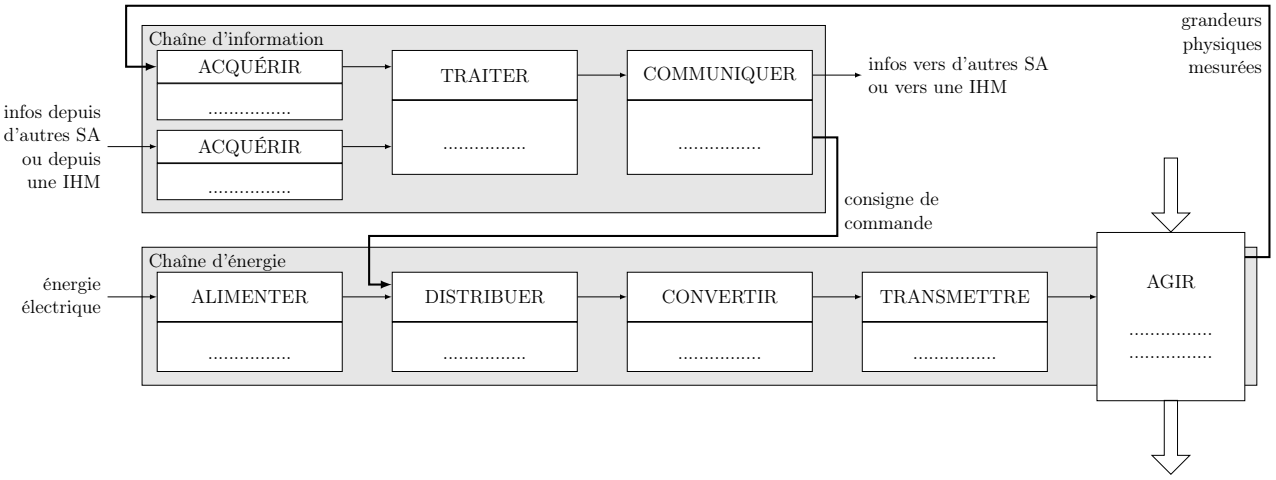
5.1 Introduction 20

5.2 Diagramme de séquence – SysML 21
5.3 Diagramme d'états – SysML 23
5.4 Diagramme d'activités – SysML (hors programme) 23

1 Préambule

Outre le dimensionnement du système en termes de performances (conception de la chaîne d'énergie), l'ingénieur doit de plus en plus concevoir la chaîne d'information qui aujourd'hui prend une part très importante dans les systèmes rencontrés.

Les capteurs acquièrent des grandeurs physiques qui sont ensuite traitées par la partie commande qui va ensuite communiquer des ordres à la chaîne d'énergie ou envoyer des informations à des éléments extérieurs.



Les objectifs de ce cours sont :

- de découvrir l'architecture des systèmes de commande actuels ;
- de manipuler l'information qui transite dans la chaîne d'information ;
- d'analyser voire de décrire le fonctionnement d'un système.

2 Architecture des systèmes de commande

2.1 Rôle d'un système de commande

Depuis le début des années 2000, les progrès technologiques sans cesse croissants des actionneurs (moteurs, vérins, etc.), des protocoles de communications numériques et des capteurs dits « intelligents » ont facilité le développement de *systèmes numériques de contrôle-commande* (SNCC) de plus en plus puissants.

Définition *Système numérique de contrôle-commande*

Un système numérique de contrôle-commande (SNCC, ou DCS pour *distributed control system* en anglais) est une structure programmable utilisée pour le pilotage d'un procédé industriel. Un SNCC est doté d'une interface homme-machine pour la supervision et d'un réseau de communication numérique pour l'interface avec les différents éléments de son environnement tels que, par exemple, les capteurs ou les autres cartes de commande associées à des sous-systèmes.

2.2 Architecture

Les SNCC sont architecturés autour d'une carte de commande principale appelée « carte mère » à laquelle sont connectées plusieurs cartes secondaires, appelées « cartes filles », gérant de manière efficace un nombre limité de tâches, par exemple l'asservissement en vitesse et position de l'arbre de sortie d'un moteur électrique.

Cette organisation modulaire est simple à concevoir et la division du traitement des informations sur plusieurs cartes présente de nombreux avantages parmi lesquels la fiabilité, les capacités de traitement et de stockage des données acquises par les capteurs ou transmises par les autres cartes, le faible temps de réponse à un événement d'entrée et la possibilité de gestion des communications par la carte mère comme par les cartes filles.

Exemple : Robot NAO

Le robot à structure humanoïde *NAO*, développé par l'entreprise française *Aldebaran Robotics*, est un exemple d'un système technique complexe. Pour gérer à la fois les fonctions de détection de l'environnement, de déplacement fluide et de préhension d'objets, le robot dispose de nombreux capteurs (sonores, visuels et tactiles), de 25 axes asservis en position et en vitesse de rotation et d'un calculateur central gérant l'ensemble des fonctions.

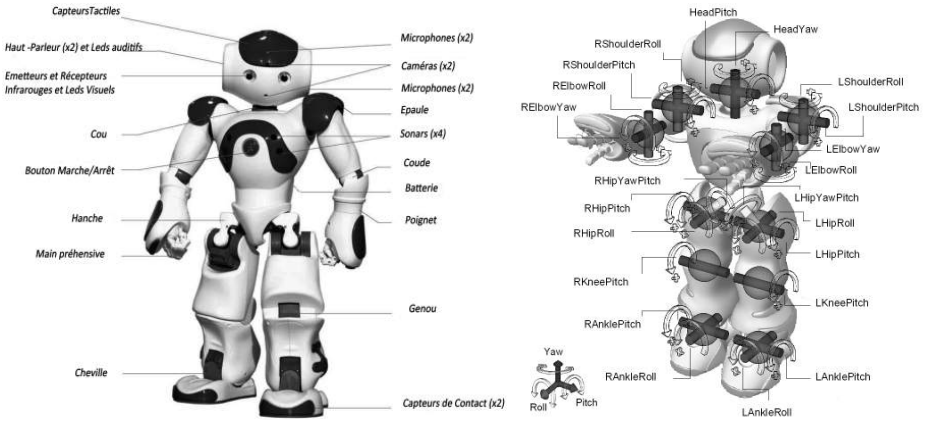


FIGURE 1 – Le robot *NAO* de l'entreprise *Aldebaran Robotics*

Lors, par exemple, du déplacement en ligne droite de ce robot humanoïde, au moins 16 axes sont sollicités (les 6 de chaque jambe + les 2 de chaque épaule afin d'assurer l'équilibre) : il est donc impossible que chacun de ces axes ait un comportement individuel non contrôlé globalement par un calculateur central. Pour améliorer les capacités du robot et gérer efficacement les flux d'informations, les 25 axes ne peuvent être gérés par un seul et même calculateur : l'utilisation d'un SNCC est donc indispensable.

Dans le cas du robot *NAO*, le SNCC est organisé autour d'une carte mère, à processeur *Intel ATOM*, qui embarque un système d'exploitation Linux (noyau 2.6 temps réel). Chacun des 25 axes est géré par une carte microcontrôleur dédiée et connectée au calculateur central par un bus de données. La programmation du robot, connecté à un ordinateur par une connexion WiFi ou Ethernet, se fait par une interface graphique reproduisant la structure d'un diagramme d'activités (voir §5.4). Le paramétrage des blocs de cette interface se fait en langage Python. La création des blocs et de nouvelles activités se fait dans de nombreux langages parmi lesquels C, C++, Python, Java et Matlab.

La réalisation d'un système de commande s'appuie sur des technologies numériques qui évoluent

très rapidement. Les performances s'améliorent chaque jour et autorisent des applications toujours plus pointues.

La commande numérique est fondamentalement constituée de portes logiques et de mémoires, qui manipulent de l'information binaire. Cette information est matérialisée électriquement, le plus souvent par un potentiel nul (0 logique) et un potentiel de quelques volts (1 logique).

2.2.1 Structure d'un FPGA

Un FPGA (*Field Programmable Gate Arrays*) est un « réseau logique programmable », c'est-à-dire une matrice de cellules logiques reliées entre elles par un réseau matriciel de connections dont les nœuds peuvent être connectés de façon programmée (Figure 2). Il existe bien entendu d'autres composants de bas niveaux mais le FPGA est une solution d'avenir relativement représentative dans sa structure et sa flexibilité en terme de conception.

L'élément de base d'un FPGA est la *cellule logique* programmable (CL), capable d'implémenter des opérations logiques élémentaires : ET, OU, NON et leurs variantes. Cette cellule logique est essentiellement composée d'une table de calcul logique réalisant l'opération et d'une bascule permettant la mémorisation d'un bit. Les FPGA contiennent plusieurs centaines de milliers de cellules logiques.

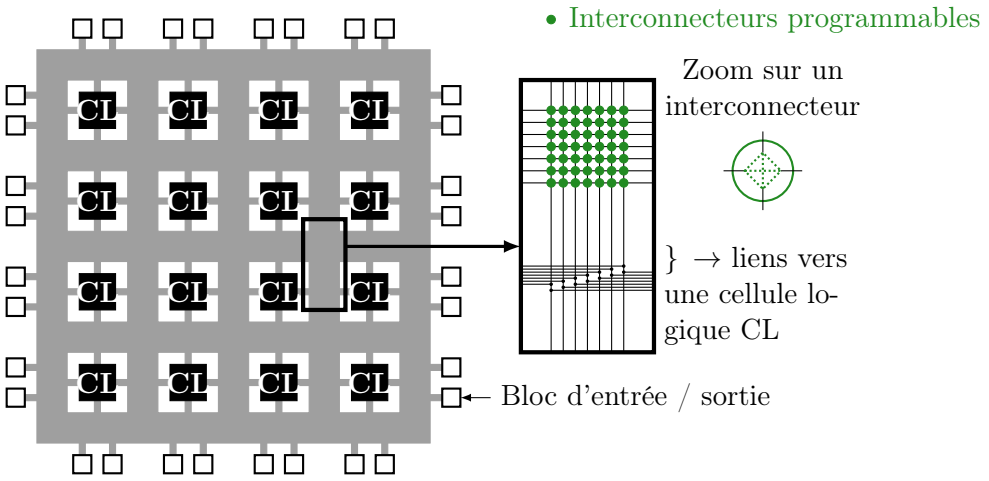


Figure 2 – Architecture interne d'un FPGA.

La matrice de routage connecte les cellules logiques entre elles et aux broches d'entrées-sorties. À chaque croisement de connecteurs, le signal peut être transmis dans toutes les directions de façon programmée. Il est ainsi possible de réaliser ces circuits logiques très complexes dans un seul composant, autorisant des vitesses d'exécutions exceptionnelles tout en consommant peu d'énergie.

Les FPGA modernes comportent, de plus, des composants périphériques comme de la mémoire, des horloges, des modules de communication série, des convertisseurs analogique-numérique ou numérique-analogique, ou encore des architectures optimisées pour accueillir une ou plusieurs structures de micro-contrôleurs dans la matrice du FPGA.

2.2.2 Structure d'un micro-contrôleur

Le cœur d'un micro-contrôleur est lui aussi constitué de portes logiques et de mémoires. À la différence du FPGA, la structure interne est déjà organisée pour réaliser un micro-processeur et ne peut

pas être reconfigurée. Le micro-contrôleur comporte de la mémoire sous forme de registres et des zones mémoire dédiées aux programmes et aux données (FIGURE 3). L'utilisateur configure les fonctions du micro-contrôleur en modifiant les registres et fixe les tâches à effectuer en implantant un programme dans la mémoire « non volatile », c'est-à-dire qu'elle ne s'efface pas lorsque le composant n'est pas alimenté. Les variables utilisées au cours de l'exécution du programme sont quant à elles stockées dans la mémoire vive « volatile » du micro-contrôleur. Un bus de données permet aux informations binaires de circuler entre les mémoires, le processeur et les périphériques d'entrée-sortie.

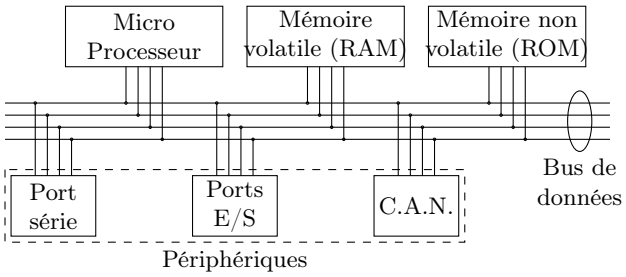


FIGURE 3 – Architecture interne d'un micro-contrôleur.

Par ailleurs, le micro-contrôleur comporte généralement des périphériques permettant de gérer des fonctionnalités avancées telles que la communication série suivant divers protocoles (communication série RS232, USB, Ethernet, etc.), la conversion analogique-numérique, les sorties PWM utilisées pour la commande de hacheurs, etc.

Le micro-contrôleur se rapproche donc du fonctionnement de la carte mère complète d'un ordinateur en un seul composant. Par comparaison avec un ordinateur de bureau, la mémoire, la vitesse d'exécution et les capacités de calcul sont nettement inférieures mais c'est généralement largement suffisant pour des applications embarquées. Sans le savoir, chacun de nous possède plusieurs dizaines (voire centaines) de micro-contrôleurs : une voiture en comporte déjà plus de cinquante (pour le contrôle de l'ABS, des injecteurs, des radars de recul, de la climatisation, de la fermeture centralisée, etc.), un ordinateur en comporte plus de vingt (contrôle de l'alimentation, de la ventilation, du lecteur CD, du clavier, de la souris, etc.) et la plupart des objets intelligents en comportent un ou plusieurs (télévision, appareil photo, téléphone, télécommande, jouet, box internet, ampoule basse consommation, etc.).

2.2.3 Les processeurs de signaux digitaux (DSP)

Alors qu'un micro-contrôleur est plutôt généraliste et d'architecture « standard », les DSP (*Digital Signal Processors*) sont des micro-contrôleurs dont l'architecture interne est optimisée pour des tâches spécifiques, comme le calcul de transformées de Fourier, le traitement d'images ou de vidéos, les protocoles de communication série de forts débits, le contrôle de moteurs brushless, etc.

2.2.4 Vers les ordinateurs embarqués et l'architecture distribuée

Les performances des micro-contrôleurs sont très variables, depuis les plus petits, peu coûteux, très sobres et dédiés à une tâche précise, jusqu'aux plus gros, disposant d'un espace mémoire confortable, de périphériques élaborés (USB, VGA, Ethernet, etc.), et pourvus d'une puissance de calcul significative (multiplication, données codées sur 32 bits, calcul flottant, etc). Sur ces derniers, il est souvent possible d'implanter un OS (*Operating System*) comme Linux.

Le micro-contrôleur propose alors la plupart des fonctionnalités de haut niveau proposées par un

ordinateur (multi-tâches, gestion des fichiers, drivers pour le pilotage de matériel, affichage sur un écran, gestion du clavier et de la souris, communication TCP-IP, etc.) tout en restant très simples à programmer à l'aide de bibliothèques standards.

Cette solution présente néanmoins des inconvénients : la gestion de l'OS absorbe une part de la capacité de calcul et conduit à une plus forte consommation, l'aspect multi-tâches rend les programmes vulnérables à des retards d'exécution lorsque le processeur est occupé à d'autres tâches et les temps de réponse sont généralement beaucoup plus longs.

Un système de commande complexe présente souvent une *architecture distribuée* où plusieurs micro-contrôleurs sont reliés entre eux par des bus de données. Les plus puissants embarquent un OS, commandent le fonctionnement global du système, et transmettent leurs consignes aux plus petits, responsables d'une tâche précise comme le contrôle d'un actionneur par exemple, et susceptibles de renvoyer des informations comme des données capteurs.

2.3 Nature des informations

Les systèmes numériques ne manipulent que des grandeurs binaires logiques donc de type « tout ou rien » : les entrées et les sorties de ses composants ne peuvent prendre que deux états.

Une succession de données binaires sera nommée grandeur numérique (ou discrète) par opposition à une grandeur analogique ou continue.

Les capteurs sont toujours utilisées pour mesurer des grandeurs physiques continues (pression, température, position,...). Ces capteurs délivrent ensuite une grandeur qui doit être échantillonnée puis quantifiée pour être utilisée par un système de commande numérique.

En général, on convertit les signaux de nature non-électrique en signaux électriques, et vice-versa, à l'aide d'un transducteur comme un microphone, un haut-parleur, une caméra numérique, un écran d'ordinateur, une antenne, un thermomètre électronique, un indicateur de débit ou un accéléromètre. Ensuite, on mesure l'intensité électrique correspondant au phénomène ou à la quantité physique pour en indiquer la valeur à un moment précis. Finalement, on convertit cette intensité électrique en un nombre pouvant être traité par le système numérique. Ce processus s'appelle la numérisation d'un signal.

3 Traitement des informations binaires

Pour représenter des valeurs chiffrées, il est nécessaire de disposer d'une base. Le système de numérotation le plus utilisé aujourd'hui est le système décimal. Les systèmes numériques utilisent plus naturellement le système binaire. La connaissance du système binaire et des changements de base binaire vers décimal et décimal vers binaire est donc essentielle à l'étude et à la réalisation des parties commandes.

3.1 Système de numération

3.1.1 Bases

Un nombre est représenté par la juxtaposition de symboles appelés digits pris parmi les éléments de la base B considérée. Les bases les plus couramment utilisées sont :

- la base 10 (ou décimale) contenant 10 symboles : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 ;
- la base 2 (ou binaire) contenant 2 symboles : 0, 1 ;
- la base 16 (ou hexadécimale) comptant 16 symboles : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F.

La figure ci-contre représente l'écriture des nombres de 0 à 15 dans 3 bases différentes.

Base 10	Base 2	Base 16
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Exemple :

$(10)_{10}$ (lire dix) représente le nombre 10 en base 10, $(10)_2$ (ou 0b10, lire un, zéro) représente le nombre 10 écrit en base 2 et $(10)_{16}$ (ou 0x10) représente le nombre 10 en base 16.

3.1.2 Changement de bases : transcodage

L'objectif est de mettre en place la correspondance entre l'expression d'un nombre dans une base (ex : base 10) et son expression dans une autre base (ex : base 2).

Passage d'une base X vers la base décimale

Un nombre $(\text{abcd})_X$ (avec a, b, c, d 4 digits) dans une base X correspond au nombre suivant en base décimale : $(\text{abcd})_X = a.X^3 + b.X^2 + c.X^1 + d.X^0$.

On parle de système de numération pondéré. On appelle poids le rang de chaque digit (exemple : a digit de poids le plus fort)

Exemple :

- Binaire vers décimal : $(1011)_2 = 11$
- Hexadécimal vers décimal : $(A3B)_{16} = 2619$

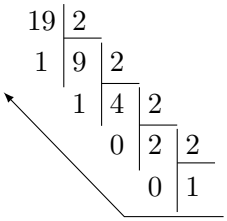
Passage de la base décimale vers une base X

On utilise la méthode des divisions successives. Soit un nombre N défini sur une base X. Son écriture en base 10 est la suivante : $(N)_X = n_p.X^p + n_{p-1}X^{p-1} + \ldots + n_1.X^1 + n_0.X^0$. Effectuer le changement de base revient à déterminer n_0, n_1, …, n_{p-1}, n_p

En mettant X en facteur il vient : $(N)_X = X(np.X^{p-1} + n_{p-1}X^{p-2} + \ldots + n_1.X^0) + n_0$ qui est de la forme : $(N)_X = X \times (quotient) + reste$. Le reste permet donc de déterminer n_0.

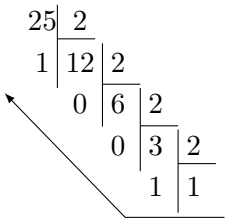
En reprenant le quotient précédent et en factorisant par X on trouve un nouveau reste n_1 et ainsi de suite jusqu'à n_p.

Ainsi pour écrire un nombre décimal en binaire, il faut faire des divisions successives par 2. Par exemple pour écrire 19 en binaire, on réalise les divisions ci-contre :

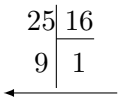


Le sens de lecture, et donc d'écriture, se fait selon la flèche. Le nombre décimal 19 s'écrit ainsi en binaire 10011, soit : $(19)_{10} = (10011)_2$.

Exemple : Écrire 25 en binaire et en hexadécimal.



$(25)_{10} = (11001)_2$



$(25)_{10} = (19)_{16}$

3.2 Codage de l'information

Les ordinateurs travaillant en binaire, tout traitement informatique ou automatique nécessite de coder l'information en binaire, à l'aide des deux états 1 ou 0. Un bit (contraction de binary digit) est un chiffre binaire (1 ou 0). Un Octet (byte en anglais) correspond à 8 bits. Un ko (kilo octet) correspond à 2^{10} chiffres soit 1024 octets.

L'écriture des nombres dans cette base donnant une multitude de chiffres, les informaticiens lui préfèrent le système hexadécimal qui permet de contracter les informations en paquets de 4 bits.

Il est également intéressant de regrouper un ensemble de valeurs binaires suivant une autre organisation qu'un système de nombres. Ces autres organisations sont appelées des **codes**. Il en existe un très grand nombre et chacun peut en créer selon un besoin spécifique.

Les qualités requises pour un code sont principalement :

- la taille du codage (nombre de bits nécessaires)
- la fiabilité de lecture
- la simplicité de manipulation

Exemple :

Les données stockées dans une carte vitale (nom, âge ...) sont codées selon une norme ASCII (American Standard Code For Information Interchange) qui nécessite 8 bits et permet de coder 256 caractères. Les chiffres de 0 à 9 en ASCII binaire sont obtenus en ajoutant 0011 devant le code binaire du chiffre sur 4 bits. Les lettres de A à Z sont codées en ajoutant 010 devant le code binaire des 26 lettres (A=01000001, etc...)



Écrire la séquence suivante en ASCII binaire : 19ans. Combien d'octets sont nécessaires pour coder cette séquence ?

0011 0001 0011 1001 0110 0001 0110 1110 0111 0011

Il y a 5 caractères, donc 5 octets (5×8 bits).

On étudie dans la suite les principaux codes à connaître. On distingue les codes pondérés pour lesquels chaque chiffre possède un poids, des codes non pondérés qui nécessitent une table de correspondance pour décoder un nombre.

3.2.1 Code binaire naturel

Ce code pondéré correspond à donner un nombre selon sa valeur en système de numérotation binaire.

C'est le seul code qui permet de réaliser des opérations (additions, soustractions, ...) très simplement.

Inconvénient : il introduit des erreurs lors d'un changement de code. Pour passer de $(01)_2$ à $(10)_2$ (donc de 1 à 2), il faut modifier les deux digits et afficher (même brièvement) $(11)_2$ (soit 3) ou $(00)_2$ (soit 0), ce qui provoque des perturbations, ou erreurs.

0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

3.2.2 Code décimal codé binaire (Code DCB)

C'est un code pondéré basé sur le code binaire naturel, mais qui est adapté à la représentation des nombres en base 10. En effet le code binaire pur n'associe pas de bits spécifiques aux unités, dizaines, centaines,.... La propriété du code DCB est d'associer quatre bits différents à chaque puissance de 10. Ainsi 1664(DCB) s'écrit 0001.0110.0110.0100 (voir tableau code binaire pur).



Ce code est utilisé sur les afficheurs 7 segments. Chaque afficheur reçoit le chiffre codé en binaire sur 4 bits.

Inconvénient : Cette représentation adaptée à la représentation binaire des nombres décimaux utilise un nombre de bits supérieur à celui du binaire naturel, et donc une place plus importante en mémoire de l'ordinateur.

> **Remarque**
>
> On peut réaliser la même typologie pour le code hexadécimal. Ainsi $(0C3F)_{16}$ s'écrit 0000.1100.0011.1111 (cette fois la représentation est optimale en mémoire).

3.2.3 Code binaire réfléchi ou code Gray

Ce code non pondéré est un arrangement du système binaire. Le passage d'un nombre à l'autre se fait en changeant l'état d'un seul bit, et celui qui change est le bit le plus à droite ne provoquant pas un nombre déjà écrit. Il est très utilisé pour décrire des automatismes (un changement d'état d'un composant correspond à un bit qui change), en particulier dans les codeurs de position absolue (FIGURE 4).

Avantage : Il apporte une garantie d'interprétation avec une erreur maximale d'une incrémentation. Inconvénient : ils sont très chers car plus difficile à réaliser que leurs homologues les codeurs incrémentaux

Pour obtenir le code Gray, il faut faire toutes les deux 2^1, quatre 2^2, huit 2^3,... lignes, une symétrie en commençant par le bit de droite et changer la valeur du bit de gauche.

	Naturel				Gray				
0	0	0	0	0	0	0	0	0	
1	0	0	0	1	0	0	0	1	1ère symétrie
2	0	0	1	0	0	0	1	1	
3	0	0	1	1	0	0	1	0	2ème symétrie
4	0	1	0	0	0	1	1	0	
5	0	1	0	1	0	1	1	1	
6	0	1	1	0	0	1	0	1	
7	0	1	1	1	0	1	0	0	3ème symétrie
8	1	0	0	0	1	1	0	0	
9	1	0	0	1	1	1	0	1	
10	1	0	1	0	1	1	1	1	
11	1	0	1	1	1	1	1	0	
12	1	1	0	0	1	0	1	0	
13	1	1	0	1	1	0	1	1	
14	1	1	1	0	1	0	0	1	
15	1	1	1	1	1	0	0	0	

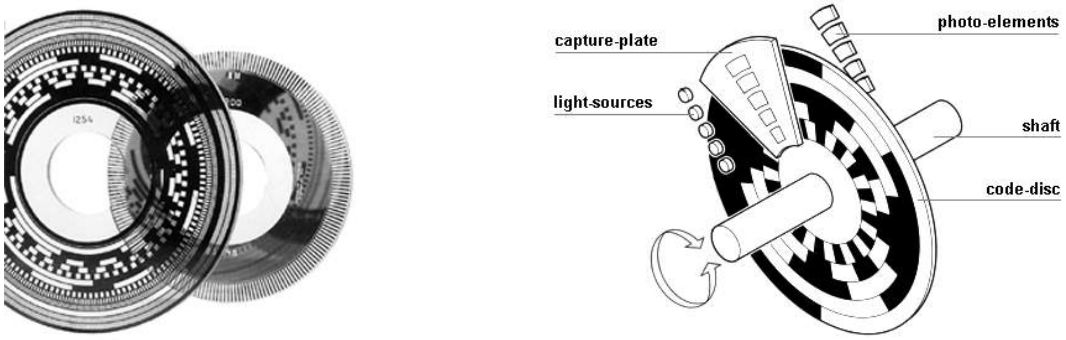
Application : Écrire en code Gray le nombre 56.

Méthode :

- Écrire 56 en binaire : 0111000.
- Multiplier le nombre par 2 (décalage vers la gauche de chaque bit) : 1110000.
- Faire un OU exclusif (somme bit à bit, sachant que 1+1=0 et 0+0=0) : 0111000+1110000 = 1001000.
- Diviser par 2 (décalage à droite de chaque bit) : soit 0100100

Exemple : Écrire 26 en code Gray.

10111



(a) Disque du codeur absolu (b) Principe du codeur absolu

FIGURE 4 – Codeur absolu

3.2.4 Code p parmi n

Le code p parmi n consiste à choisir parmi n bits, p bits égaux à 1 et $n-p$ bits égaux à 0. Le nombre de combinaisons répondant à cette définition est égal à $C_n^p = \dfrac{n!}{p!(n_p)!}$.

Par exemple, le tableau ci-contre présente un code 3 parmi 5 et met en correspondance la valeur décimale et son écriture en code 3 parmi 5. Ce code comporte trois « 1 » et deux « 0 » et permet 10 combinaisons : $C_5^3 = \dfrac{5!}{3!(5-3)!} = 10$.

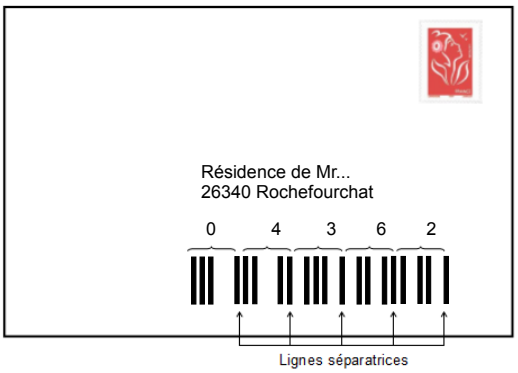
Décimal	Code 3 parmi 5				
0	0	0	1	1	1
1	0	1	0	1	1
2	0	1	1	0	1
3	0	1	1	1	0
4	1	0	0	1	1
5	1	0	1	0	1
6	1	0	1	1	0
7	1	0	0	0	1
8	1	1	0	1	0
9	1	1	1	0	0

L'avantage du code p parmi n est qu'il peut être personnalisé. En effet il existe $C_n^p!$ arrangements de la codification. Ainsi le code 3 parmi 5 procure $C_5^3! = 10!$ arrangements différents, soit 3 628 800 possibilités.

L'inconvénient de ce code non pondéré est qu'il est nécessaire d'avoir la table de correspondance pour décoder un nombre.

Application : Tri postal

Le tri automatique des plis postaux nécessite la codification numérique du bureau distributeur. Le code numérique est lu automatiquement (lecture O.C.R. : Reconnaissance Optique de Caractères) ou par un opérateur. Il est ensuite traduit en un code à barres qui est matérialisé sous la forme de bâtonnets déposés sur le pli postal. Les « 1 » sont codés par des bâtonnets foncés, les « 0 » sont matérialisés par des bâtonnets pratiquement transparents, ces « 1 » et ces « 0 » étant séparés par des espaces. Le code 3 parmi 5 retenu pour le code postal est celui donné précédemment.



Exemple : Indiquer le code postal correspondant au code barre suivant : 

05619

4 Logique combinatoire

4.1 Définitions

4.1.1 Variables binaires

De nombreux composants utilisés en automatisme ne peuvent normalement prendre que deux états différents : lampe allumée ou éteinte, bouton-poussoir actionné ou relâché, moteur tournant ou à l'arrêt, vérin pneumatique sorti ou rentré...

A chacun de ces composants, on peut associer une **variable binaire** (ou logique, ou Tout Ou Rien) qui ne peut prendre que deux valeurs notées 0 ou 1 (vrai ou faux, oui ou non).

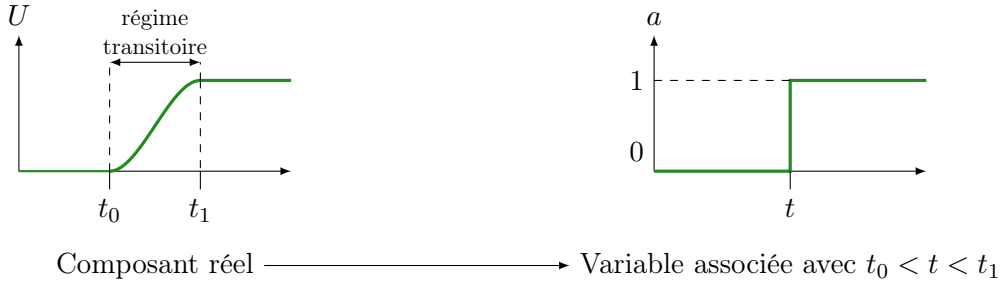
Définition *Variable binaire*

Une variable a est binaire si et seulement si elle ne peut prendre, à chaque instant, qu'une seule valeur parmi un ensemble de 2 valeurs possibles.

☞ **Support de cours :** Q1

Remarque

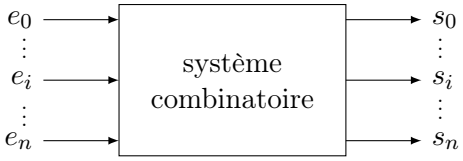
- le comportement « Tout ou Rien » (TOR) ne correspond qu'au comportement normalement prévu en régime stabilisé et en l'absence de tout dysfonctionnement.
- l'association d'une variable binaire à un composant ne peut pas rendre compte des états transitoires apparaissant entre deux états stables. C'est donc une simplification du comportement réel.



4.1.2 Système binaire

Définition *Système binaire*

Un système est dit binaire ou logique si les variables d'entrée et de sortie sont binaires.



4.1.3 Systèmes combinatoire et séquentiel

Définition *Système binaire*

Un système logique combinatoire est un système binaire pour lequel à un état des variables d'entrée e_i correspond un unique état des variables de sortie s_j. (La réciproque n'est pas vraie)

La FIGURE 5a illustre l'appuie sur un interrupteur à deux positions :

- sur la position 0 (entrée à 0), la lumière est éteinte (sortie à 0),
- sur la position 1 (entrée à 1), la lumière est allumée (sortie à 1).

Définition *Système séquentiel*

En revanche, un système logique est dit séquentiel si les sorties s_j ne dépendent pas uniquement des entrées e_i mais aussi de l'évolution antérieure du système (historique).

En effet, à un état des e_i peuvent alors correspondre plusieurs états des s_j. Il y a alors nécessité de faire apparaître de nouvelles variables internes (appelées mémoires).

La FIGURE 5b illustre l'appuie sur un interrupteur de type bouton poussoir :

- sur la position 0 (entrée à 0), la lumière est éteinte (sortie à 0),
- sur la position 1 (entrée à 1, appui sur le bouton), la lumière s'allume (sortie à 1),
- sur la position 0 (entrée à 0, relâchement du bouton), la lumière reste allumée (sortie à 0),
- sur la position 1 (entrée à 1, appui sur le bouton), la lumière s'éteint (sortie à 0).



(a) Système logique combinatoire (b) Système logique séquentielle

FIGURE 5 – Différence entre système logique combinatoire et séquentiel

4.2 Algèbre de Boole

Pour traiter les systèmes combinatoires, on fait appel à l'algèbre de Boole.

Soit un ensemble $B = \{0, 1\}$ constitué de deux éléments représentant les deux états logiques vrai ou faux. Cet ensemble est muni d'une structure d'algèbre avec les trois lois suivantes :

Définition *Complémentarité : **NON***

$$B \longrightarrow B$$
$$a \longmapsto \text{NON}(a) = \overline{a}$$

$\overline{a}$ est à 1 si a est à 0 et réciproquement (voir table 6a).

Définition *Produit booléen : **ET***

$$B \times B \longrightarrow B$$
$$(a, b) \longmapsto a \text{ ET } b = a \cdot b$$

$a \cdot b$ est à 1 si et seulement si a est à 1 ET b est à 1 (voir table 6b).

Définition *Somme booléenne :* ***OU***

$$B \times B \longrightarrow B$$
$$(a, b) \longmapsto a \text{ OU } b = a + b$$

$a + b$ est à 1 si et seulement si a est à 1 OU b est à 1 (voir table 6c).
Deux expressions logiques A et B équivalentes seront notés avec le symbole « = » : $A = B$.

Propriété *Lois ET et OU*

	Loi OU	**Loi ET**
Commutativité	$a + b = b + a$	$a \cdot b = b \cdot a$
Distributivité	$a \cdot (b + c) = a \cdot b + a \cdot c$	$(a \cdot b) + c = (a + c) \cdot (b + c)$
Associativité	$a + (b + c) = (a + b) + c = a + b + c$	$a \cdot (b \cdot c) = (a \cdot b) \cdot c = a \cdot b \cdot c$
Élément neutre 0	$a + 0 = a$	$a \cdot 0 = 0$
Élément neutre 1	$a + 1 = 1$	$a \cdot 1 = a$
Complémentarité	$a + \overline{a} = 1$	$a \cdot \overline{a} = 0$
Idempotence	$a + a = a$	$a \cdot a = a$

D'autres propriétés algébriques permettent d'aider à la simplification des expressions :

- double complément : $\overline{\overline{a}} = a$
- absoption : $a + a \cdot b = a$
- inclusion : $a + \overline{a} \cdot b = a + b$

Définition *Théorèmes de De Morgan*

Le complément d'un OU est le ET des compléments : $\overline{a + b} = \overline{a}.\overline{b}$

Le complément d'un ET est le OU des compléments : $\overline{a \cdot b} = \overline{a} + \overline{b}$

Exemple :

Déterminer les expressions complémentaires $\overline{P}$ et $\overline{Q}$ des expressions logiques suivantes : $P = x \cdot y \cdot (z + \overline{t})$ et $Q = \overline{x} \cdot \overline{y} + y + x \cdot t$.

$\overline{P} = \overline{x \cdot y \cdot (z + \overline{t})} = \overline{x} + \overline{y} + (\overline{z + \overline{t}}) = \overline{x} + \overline{y} + \overline{z} \cdot t$

$\overline{Q} = \overline{\overline{x} \cdot \overline{y} + y + x \cdot t} = x + y \cdot \overline{y} \cdot \overline{x} + \overline{t}$

4.3 Représentation des fonctions logiques

Pour définir une fonction logique associée à un processus, il existe plusieurs représentations possibles :

- représentation algébriques : $f(a, b, c, d) = a \cdot b \cdot \overline{c} + b \cdot c \cdot \overline{d}$;
- représentation graphique : table de vérité, logigramme, chronogramme.

Remarque

Il existe d'autres représentations graphiques non traitées dans ce paragraphe.

4.3.1 Table de vérité

Définition *Table de vérité*

La table de vérité d'une fonction logique est un tableau comportant une colonne par variable logique et une colonne pour la fonction.

Le nombre de lignes est donné par le nombre de variables de la fonction et représente le nombre de combinaisons possibles de l'état de ces variables. Si la fonction dépend de n variables logiques, alors la table de vérité comportera 2^n lignes.

La Figure 6 donne les tables de vérités des fonctions logiques de l'algèbre de Boole.

a	$\overline{a}$
0	1
1	0

(a) Table de la loi NON

a	b	$a \cdot b$
0	0	0
0	1	0
1	0	0
1	1	1

(b) Table de la loi ET

a	b	$a + b$
0	0	0
0	1	1
1	0	1
1	1	1

(c) Table de la loi OU

Figure 6 – Table de vérité des lois de l'algèbre de Boole

Exemple : Compléter les tables de vérité du théorème de De Morgan

a	b	$\overline{a}$	$\overline{b}$	$\overline{a \cdot b}$	$\overline{a} + \overline{b}$	$\overline{a + b}$	$\overline{a} \cdot \overline{b}$
0	0	1	1	1	1	1	1
0	1	1	0	1	1	0	0
1	1	0	0	0	0	0	0
1	0	0	1	1	1	0	0

4.3.2 Équations logiques

Il est possible à l'aide de la table de vérité de définir une équation donnant les variables de sortie en fonction des variables d'entrée.

Il existe 2 modes d'écriture particuliers :

- somme canonique. (somme de produits) : $(a \cdot b) + (c \cdot d)$
- produit canonique (produit de sommes) : $(a + b) \cdot (c + d)$

Méthode de détermination de la somme canonique : pour chaque ligne où la sortie vaut 1, déterminer la combinaison d'entrées correspondante à l'aide de l'opérateur ET puis sommer ces combinaisons.

a	b	S
0	0	0
0	1	1
1	0	0
1	1	1

$S = 1$ quand $a = 0$ ET $b = 1$ soit quand on a l'expression $\overline{a} \cdot b$ vraie.

$S = 1$ quand $a = 1$ ET $b = 1$ soit quand on a l'expression $a \cdot b$ vraie.

Au final, on a $\boxed{S = \overline{a} \cdot b + a \cdot b}$.

☞ **Support de cours :** Q2 et Q3

4.3.3 Logigrammes

Définition *Logigramme*

Un logigramme est une représentation graphique d'une fonction logique utilisant des composants élémentaires appelés « portes logiques ».

Ces représentations sont particulièrement utilisées dans les documentations techniques des composants électroniques.

Le Tableau 2 donne les principaux symboles logiques dans les normes européennes et américaines.

☞ **Support de cours :** Q4

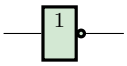
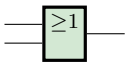
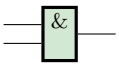
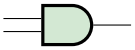
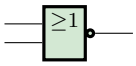
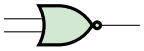
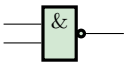
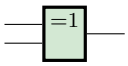
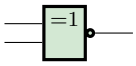
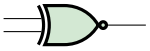
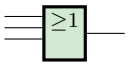
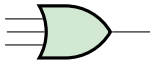
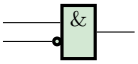
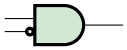
Fonction logique	Symbole européen	Symbole américain
NON $\overline{a}$		
OU $a+b$		
ET $a \cdot b$		
NON OU $\overline{a+b}$		
NON ET $\overline{a \cdot b}$		
OU EXCLUSIF $a \oplus b = \overline{a} \cdot b + a \cdot \overline{b}$		
ET EXCLUSIF / IDENTITÉ $a \otimes b = \overline{a \oplus b} = \overline{a} \cdot \overline{b} + a \cdot b$		
Il peut y avoir plus de deux entrées sur une porte logique : OU à 3 entrées		
Il peut y avoir des entrées inhibées (l'entrée est complémentée) : ET à 2 entrées la 2nde est complémentée		

TABLE 2 – Tableau des portes logiques.

4.3.4 Chronogramme

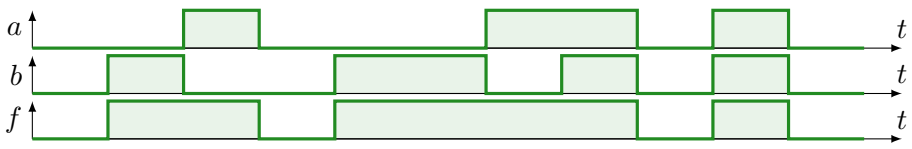
Une autre représentation classique des variables et fonctions logiques est le chronogramme.

Définition *Chronogramme*

Un chronogramme est une représentation de l'évolution temporelle d'une variable ou une fonction logique en fonction du temps.

Exemple :

Soit la fonction logique $f(a,b) = a + b$. La figure suivante donne l'évolution de f en fonction du temps.



Remarque

On s'aide d'un chronogramme pour savoir si un système est combinatoire ou séquentiel. Il suffit de regarder si pour un même état des variables d'entrée, les sorties sont toujours les mêmes (cf. exemple de l'interrupteur).

Notion de fronts montant et descendant Les fronts permettent de représenter le changement d'état d'une variable. Cette notion est très importante dans le cadre de la synchronisation des processus mais aussi la prise en compte de l'évolution du système à partir de capteurs (exemple appui sur un bouton poussoir, codeur incrémental...).

Définition

Le front montant d'une variable est vrai à l'instant pour lequel la variable passe de son état logique 0 à son état logique 1. Il est noté $\uparrow a$.

Le front descendant d'une variable est vrai à l'instant pour lequel la variable passe de son état logique 1 à son état logique 0. Il est noté $\downarrow a$.

Un front est d'une durée théorique nulle. Il sera représenté par une impulsion de Dirac sur un chronogramme.

Exemple :

Soit la variable logique b. La figure suivante donne l'évolution de b en fonction du temps et l'évolution des fronts montant et descendant de b.



5 Logique séquentielle

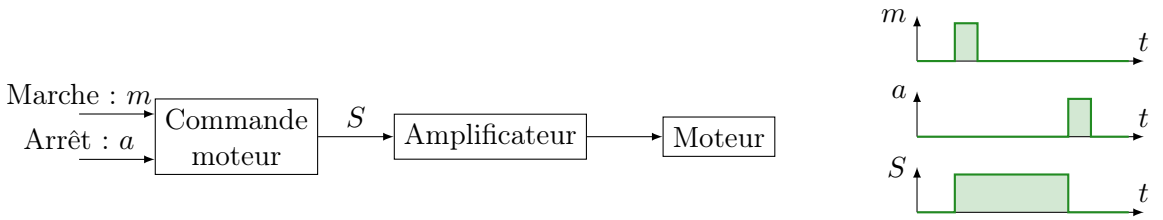
5.1 Introduction

Définition *Système séquentiel*

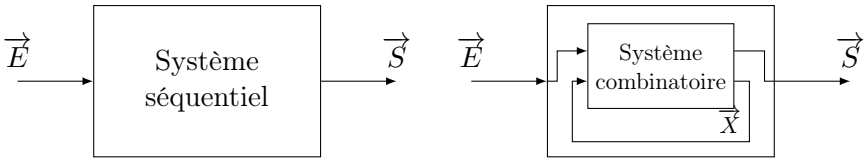
Un système séquentiel est un système dont les sorties S_i dépendent de l'histoire des entrées $E_i(t)$.

Exemple : Commande d'un moteur électrique par un système séquentiel

On remarque sur le chronogramme de droite que la sortie S peut présenter une valeur différente (0 ou 1) pour une configuration identique des entrées m et a.



Le système est capable de mémoriser de l'information. Cette information mémorisée est l'état du système, qui peut être représenté par un vecteur d'état booléen : $\overrightarrow{X} = (x_1, x_2, \ldots, x_n)$.



Connaissant $\overrightarrow{X}$ et $\overrightarrow{E}$ (le vecteur d'état booléen des entrées), la sortie $\overrightarrow{S}$ peut être déterminée comme une fonction booléenne de $\overrightarrow{X}$ et de $\overrightarrow{E}$: $\overrightarrow{S} = f(\overrightarrow{X}, \overrightarrow{E})$ (donc via un système combinatoire).

L'état interne $\overrightarrow{X}$ à l'instant t dépend de $\overrightarrow{E}(t)$ et de l'état interne immédiatement précédent : $\overrightarrow{X}(t) = g(\overrightarrow{E}(t), \overrightarrow{X}(t - \Delta t))$.

Les tables de vérité ou logigrammes ne permettent plus de décrire l'évolution de S en fonction de E. On peut utiliser alors les chronogrammes (aussi appelés diagrammes de Gantt).

Pour décrire le fonctionnement global du système, de la partie logicielle, on utilise plutôt des outils de description du SysML : les diagrammes de séquence, d'états ou d'activité.

5.2 Diagramme de séquence – SysML

Ce diagramme permet de définir la chronologie des interactions entre les acteurs (utilisateurs) et le système. Il permet de décrire un scénario correspondant à un cas d'utilisation.

Le principe est de décrire les messages envoyés entre les acteurs dans l'ordre chronologique (le temps s'écoule vers le bas). Le comportement interne des constituants n'est pas détaillé dans ce diagramme.

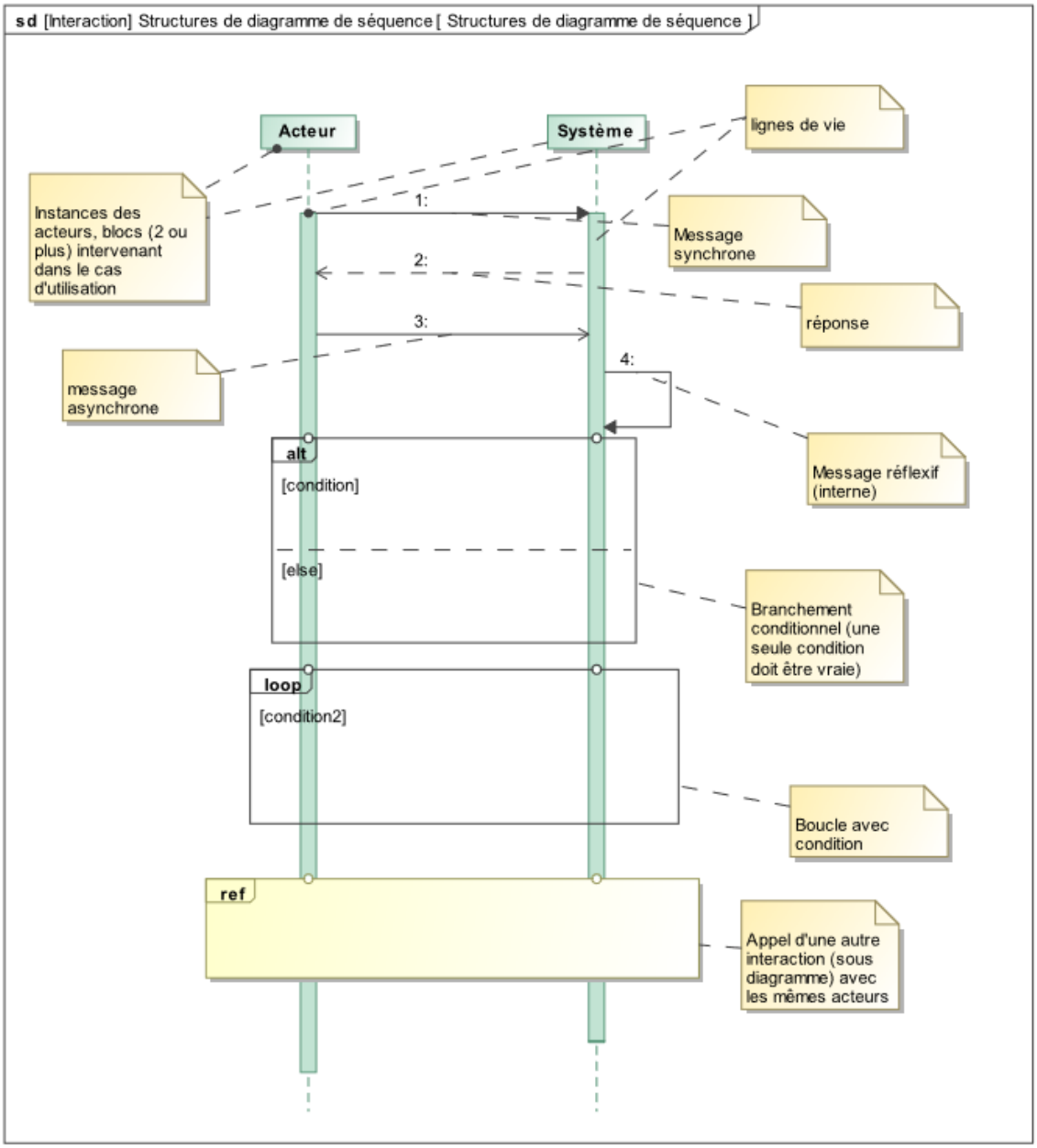


FIGURE 7 – Structure d'un diagramme de séquence.

Les éléments constitutifs du diagramme sont (FIGURE 7) :

- les lignes de vie qui représentent les acteurs (du diagramme de cas d'utilisation) participant à la communication. Elles peuvent être actives ou inactives (bandes verticales).
- les messages : éléments de communication unidirectionnels entre les lignes. Il existe plusieurs types de messages :
  - ◇ Message synchrone : l'émetteur est bloqué en attente de réponse
  - ◇ Message asynchrone : pas de réponse attendue
  - ◇ Message réflexif : comportement interne
- les opérateurs qui permettent de réaliser des tests, des opérations en parallèle, des boucles...
  - ◇ **loop** (boucle) : plusieurs exécutions successive d'un même fragment,
  - ◇ **opt** (optionnel) : exécution si la condition fournie est vraie

- ◇ **alt** (fragments alternatifs) : éxecution du fragment si une condition parmi plusieurs est vraie
- ◇ **ref** : appel d'un autre diagramme de séquence défini ailleurs
- ◇ **par** : exécution en parallèle

☞ **Support de cours :** Lire le diagramme de séquence du support de cours.

5.3 Diagramme d'états – SysML

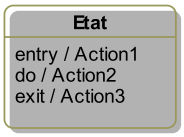
Ce diagramme permet de décrire les différents états que peut traverser le système en fonction des évènements perçus.

Il est utilisé essentiellement pour décrire les modes de marche, d'arrêt, d'attente d'un système ou sous-système.

Il utilise plusieurs symboles particuliers :

- un état représenté par un bloc qui représente une situation d'une durée finie. Pendant cet état, il peut se produire des actions. L'état initial dans lequel est placé le diagramme au départ est représenté par un symbole particulier ●. On peut également utiliser un symbole pour représenter le ou les états finaux (le diagramme ne peut plus évoluer) ◉.
- les événements représentés par des flèches permettent de décrire comment passer d'un état à un autre. On peut associer une condition booléenne (appelée condition de garde) à l'événement ou utiliser des mots clés particuliers :
  - ◇ `after x s` : après x secondes on passe à l'état suivant,
  - ◇ `at h` : à l'heure indiquée, on passe à l'état suivant
  - ◇ `when x=xc` : on passe à l'état suivant quand une valeur atteint une valeur donnée
  - ◇ si aucun texte n'est noté sur la flèche (on parle de transition de complétion), ceci indique que l'on passe à l'état suivant dès que les actions associées à un état sont terminées.

Il peut être utile de spécifier à l'intérieur d'un état des actions qui sont effectuées. On utilise pour cela le mot clé `do` suivi d'une ou plusieurs actions. On peut spécifier plus précisément à quel moment se feront les actions à l'intérieur d'un état :



- `entry` suivi d'une action indique que l'action est faite en entrant dans l'état
- `exit` suivi d'une action précise que l'action est faite en sortie de l'état

Un état peut contenir lui même un diagramme d'état. On parle d'état composé ou composite. Comme tout diagramme d'état, l'état composite contiendra une étape initiale qui sera activée lorsqu'on entrera dans l'état composite. On le représente à l'intérieur d'un bloc ou bien on utilise un symbole pour faire référence au diagramme d'état.

☞ **Support de cours :** Lire le diagramme d'état du support de cours

☞ **Support de cours :** Réaliser le diagramme d'états demandé

5.4 Diagramme d'activités – SysML (hors programme)

Ce diagramme permet de décrire le déroulement séquentiel d'un processus en spécifiant les actions, activités ou tâches réalisées. Il peut aussi préciser ce qui est produit, consommé ou transformé au cours de l'activité. Il est très utile pour décrire précisément ce qui se passe dans un état donné de diagrammes d'états.

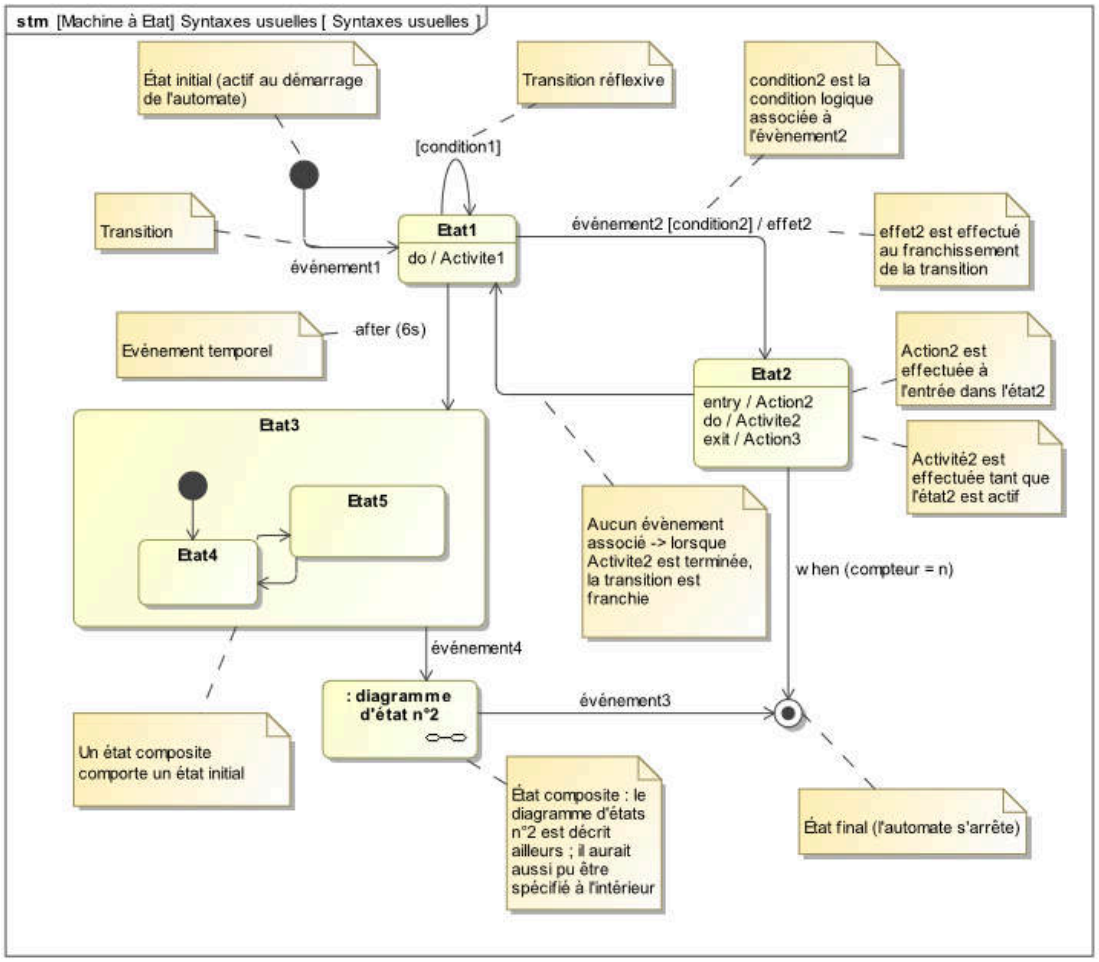


FIGURE 8 – Structures usuelles d'un diagramme d'état

Dans sa forme la plus restreinte ce diagramme correspond à un algorigramme et peut directement être traduit en programme informatique.

Il utilise les mêmes symboles de base que le diagramme d'état (blocs et flèches). Cependant, aucune condition n'est indiqué sur la flèche car celle-ci représente un signal émis dès que les actions associées à un bloc sont terminées.

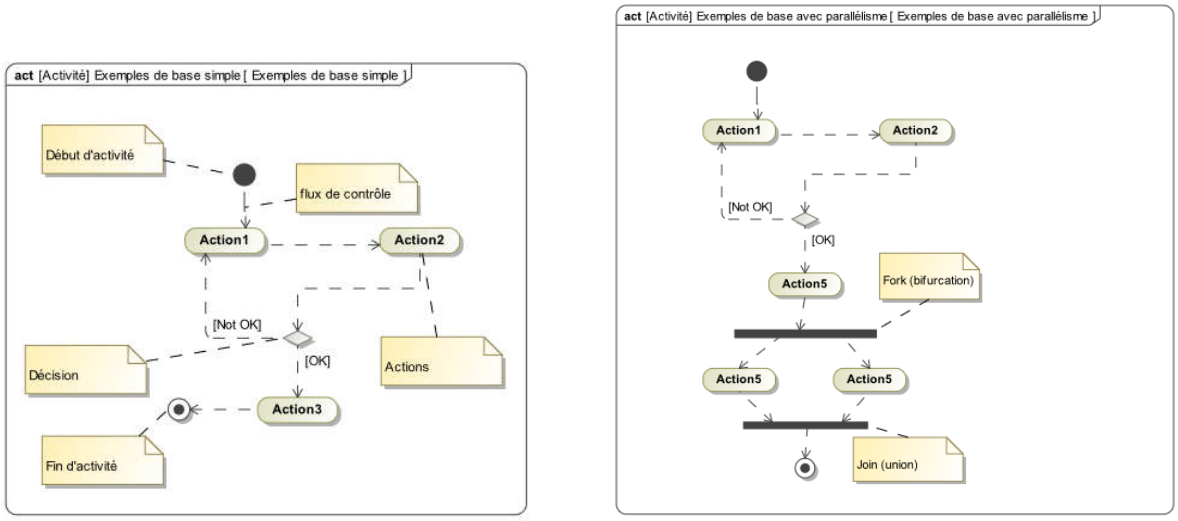
Pour représenter un choix à faire entre plusieurs conditions exclusives, on utilise le symbole de branchement conditionnel. Les conditions sont notées entre crochets en sortie du branchement conditionnel (FIGURE 9a).

Si des actions doivent se faire en parallèle ou doivent être synchronisées, on utilise des symboles particuliers appelés Fork et Join (FIGURE 9b).



Remarque

Le diagramme d'activité est plus général qu'un algorigramme car non seulement il présente les flux de contrôle mais également les flux de matière, énergie, information qui sont modifiés par le système. Il représente ainsi une spécification plus précise de ce que l'on décrit par des diagrammes IBD.



(a) Exemple avec 1 choix (b) Exemple avec parallèlisme

FIGURE 9 – Diagramme d'activité – Algorigramme

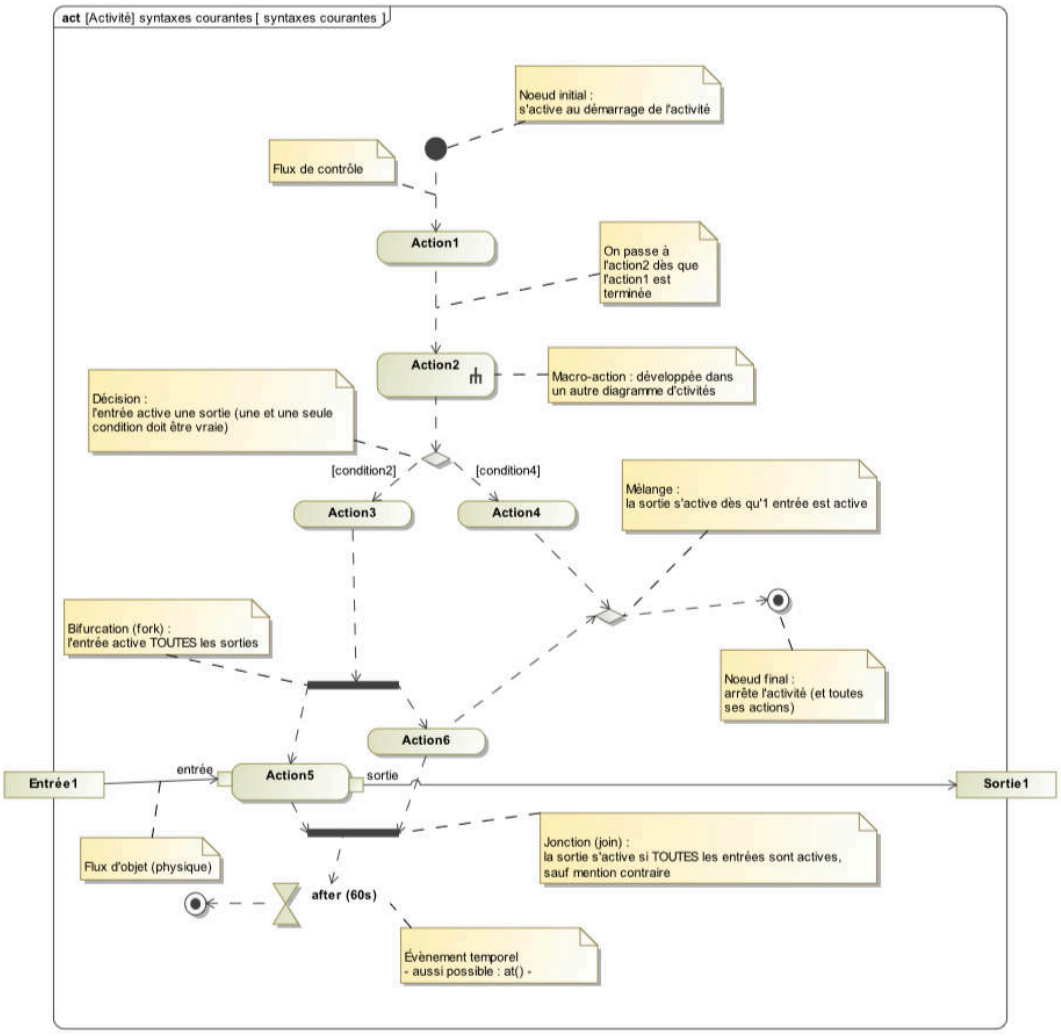


FIGURE 10 – Structures usuelles d'un diagramme d'activités.